boost::asio之socket的创建和连接

Posted on 2023-02-13 16:29:29 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2LfzYBkRCfdEDrtE6hWz8VrCLoS)

网络编程基本流程

网络编程的基本流程对于服务端是这样的
服务端
1）socket——创建socket对象。

2）bind——绑定本机ip+port。

3）listen——监听来电，若在监听到来电，则建立起连接。

4）accept——再创建一个socket对象给其收发消息。原因是现实中服务端都是面对多个客户端，那么为了区分各个客户端，则每个客户端都需再分配一个socket对象进行收发消息。

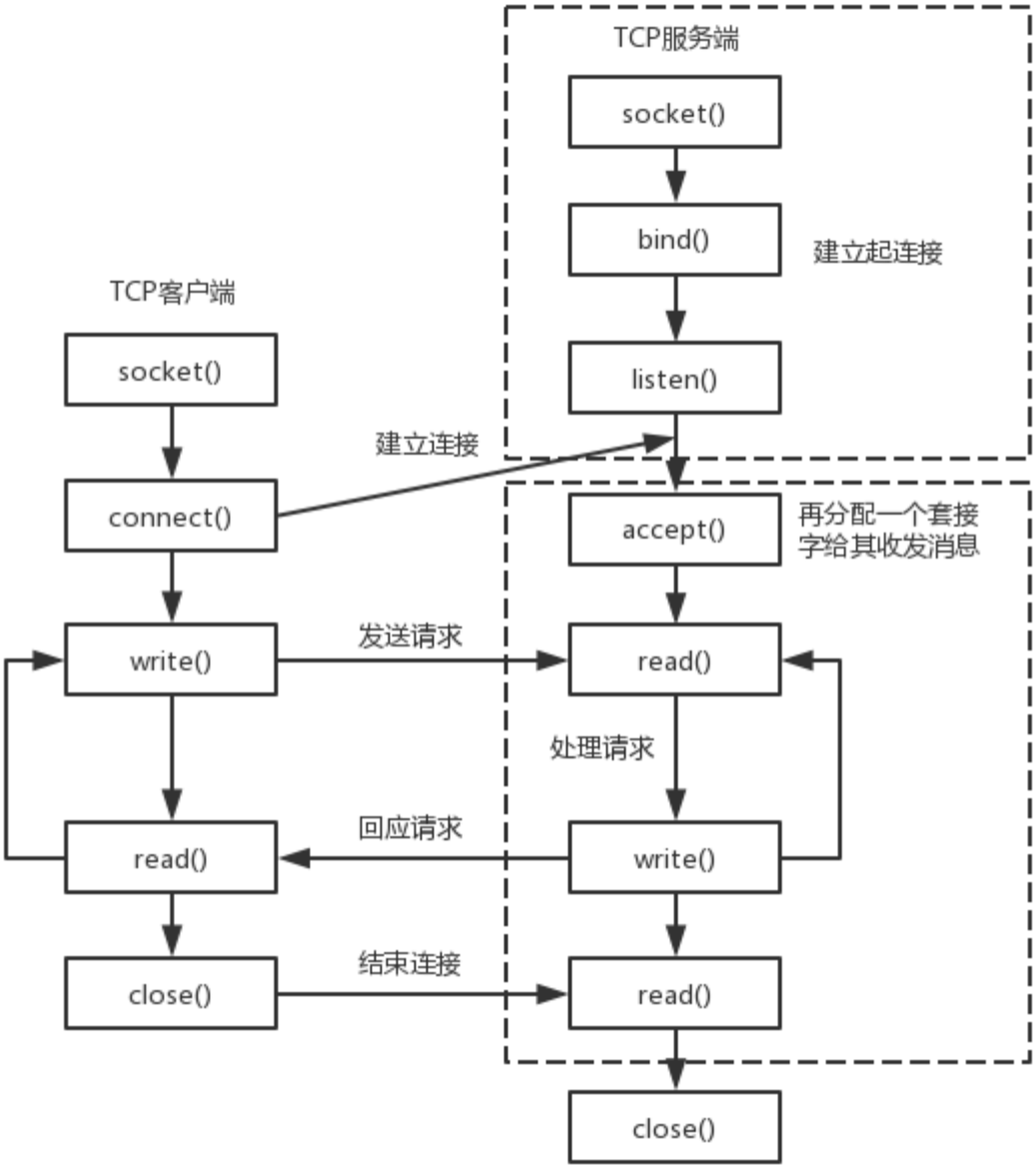
5）read、write——就是收发消息了。

对于客户端是这样的
客户端
1）socket——创建socket对象。

2）connect——根据服务端ip+port，发起连接请求。

3）write、read——建立连接后，就可发收消息了。

图示如下



相关的网络编程技术可以看看我之前写的文章
https://llfc.club/articlepage?id=2LXIKWJtKGblnWtHT7TplLKK6ze (https://llfc.club/articlepage?id=2LXIKWJtKGblnWtHT7TplLKK6ze)
接下来按照上述流程，我们用boost::asio逐步介绍。

终端节点的创建

所谓终端节点就是用来通信的端对端的节点，可以通过ip地址和端口构造，其的节点可以连接这个终端节点做通信.

如果我们是客户端，我们可以通过对端的ip和端口构造一个endpoint，用这个endpoint和其通信。

```
1. int  client_end_point() {
2.     // Step 1. Assume that the client application has already
3.     // obtained the IP-address and the protocol port number.
4.     std::string raw_ip_address = "127.0.0.1";
5.     unsigned short port_num = 3333;
6.
7.     // Used to store information about error that happens
8.     // while parsing the raw IP-address.
9.     boost::system::error_code ec;
10.    // Step 2. Using IP protocol version independent address
11.    // representation.
12.    asio::ip::address ip_address =
13.        asio::ip::address::from_string(raw_ip_address, ec);
14.
15.    if (ec.value() != 0) {
16.        // Provided IP address is invalid. Breaking execution.
17.        std::cout
18.            << "Failed to parse the IP address. Error code = "
19.            << ec.value() << ". Message: " << ec.message();
20.        return ec.value();
21.    }
22.
23.    // Step 3.
24.    asio::ip::tcp::endpoint ep(ip_address, port_num);
25.
26.    // Step 4. The endpoint is ready and can be used to specify a
27.    // particular server in the network the client wants to
28.    // communicate with.
29.
30.    return 0;
31. }
```

如果是服务端，则只需根据本地地址绑定就可以生成endpoint

```
int  server_end_point(){
    // Step 1. Here we assume that the server application has
    //already obtained the protocol port number.
    unsigned short port_num = 3333;

    // Step 2. Create special object of asio::ip::address class
    // that specifies all IP-addresses available on the host. Note
    // that here we assume that server works over IPv6 protocol.
    asio::ip::address ip_address = asio::ip::address_v6::any();

    // Step 3.
    asio::ip::tcp::endpoint ep(ip_address, port_num);

    // Step 4. The endpoint is created and can be used to
    // specify the IP addresses and a port number on which
    // the server application wants to listen for incoming
    // connections.

    return 0;
}
```

创建socket

创建socket分为4步，创建上下文iocontext，选择协议，生成socket，打开socket。

```
1. int create_tcp_socket() {
2.     // Step 1. An instance of 'io_service' class is required by
3.         // socket constructor.
4.     asio::io_context  ios;
5.
6.     // Step 2. Creating an object of 'tcp' class representing
7.     // a TCP protocol with IPv4 as underlying protocol.
8.     asio::ip::tcp protocol = asio::ip::tcp::v4();
9.
10.    // Step 3. Instantiating an active TCP socket object.
11.    asio::ip::tcp::socket sock(ios);
12.
13.    // Used to store information about error that happens
14.    // while opening the socket.
15.    boost::system::error_code ec;
16.
17.    // Step 4. Opening the socket.
18.    sock.open(protocol, ec);
19.
20.    if (ec.value() != 0) {
21.        // Failed to open the socket.
22.        std::cout
23.            << "Failed to open the socket! Error code = "
24.            << ec.value() << ". Message: " << ec.message();
25.        return ec.value();
26.    }
27.
28.    return 0;
29. }
```

上述socket只是通信的socket，如果是服务端，我们还需要生成一个acceptor的socket，用来接收新的连接。

```
int  create_acceptor_socket() {
    // Step 1. An instance of 'io_service' class is required by
        // socket constructor.
    asio::io_context ios;

    // Step 2. Creating an object of 'tcp' class representing
            // a TCP protocol with IPv6 as underlying protocol.
    asio::ip::tcp protocol = asio::ip::tcp::v6();

    // Step 3. Instantiating an acceptor socket object.
    asio::ip::tcp::acceptor acceptor(ios);

    // Used to store information about error that happens
    // while opening the acceptor socket.
    boost::system::error_code ec;

    // Step 4. Opening the acceptor socket.
    acceptor.open(protocol, ec);

    if (ec.value() != 0) {
        // Failed to open the socket.
        std::cout
            << "Failed to open the acceptor socket!"
            << "Error code = "
            << ec.value() << ". Message: " << ec.message();
        return ec.value();
    }

    return 0;
}
```

绑定acceptor

对于acceptor类型的socket，服务器要将其绑定到指定的断点,所有连接这个端点的连接都可以被接收到。

```
int  bind_acceptor_socket() {

    // Step 1. Here we assume that the server application has
       // already obtained the protocol port number.
    unsigned short port_num = 3333;

    // Step 2. Creating an endpoint.
    asio::ip::tcp::endpoint ep(asio::ip::address_v4::any(),
       port_num);

    // Used by 'acceptor' class constructor.
    asio::io_context  ios;

    // Step 3. Creating and opening an acceptor socket.
    asio::ip::tcp::acceptor acceptor(ios, ep.protocol());

    boost::system::error_code ec;

    // Step 4. Binding the acceptor socket.
    acceptor.bind(ep, ec);

    // Handling errors if any.
    if (ec.value() != 0) {
       // Failed to bind the acceptor socket. Breaking
       // execution.
       std::cout << "Failed to bind the acceptor socket."
          << "Error code = " << ec.value() << ". Message: "
          << ec.message();

       return ec.value();
    }

    return 0;
}
```

连接指定的端点

作为客户端可以连接服务器指定的端点进行连接

```
int  connect_to_end() {
    // Step 1. Assume that the client application has already
        // obtained the IP address and protocol port number of the
        // target server.
    std::string raw_ip_address = "127.0.0.1";
    unsigned short port_num = 3333;

    try {
        // Step 2. Creating an endpoint designating
        // a target server application.
        asio::ip::tcp::endpoint
            ep(asio::ip::address::from_string(raw_ip_address),
                port_num);

        asio::io_context ios;

        // Step 3. Creating and opening a socket.
        asio::ip::tcp::socket sock(ios, ep.protocol());

        // Step 4. Connecting a socket.
        sock.connect(ep);

        // At this point socket 'sock' is connected to
        // the server application and can be used
        // to send data to or receive data from it.
    }
    // Overloads of asio::ip::address::from_string() and
    // asio::ip::tcp::socket::connect() used here throw
    // exceptions in case of error condition.
    catch (system::system_error& e) {
        std::cout << "Error occured! Error code = " << e.code()
            << ". Message: " << e.what();

        return e.code().value();
    }
}
```

服务器接收连接

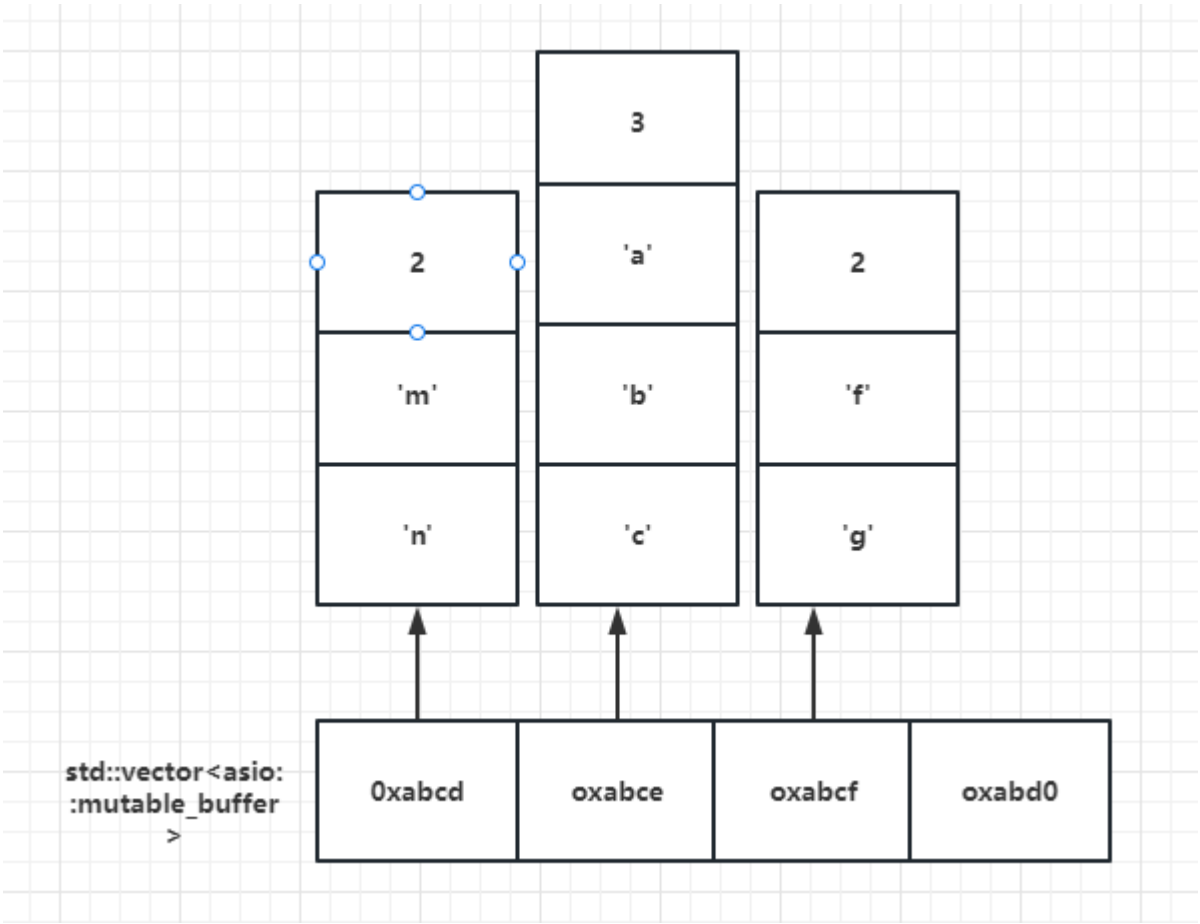
当有客户端连接时，服务器需要接收连接

```
1. int accept_new_connection(){
2.     // The size of the queue containing the pending connection
3.         // requests.
4.     const int BACKLOG_SIZE = 30;
5.
6.     // Step 1. Here we assume that the server application has
7.     // already obtained the protocol port number.
8.     unsigned short port_num = 3333;
9.
10.    // Step 2. Creating a server endpoint.
11.    asio::ip::tcp::endpoint ep(asio::ip::address_v4::any(),
12.       port_num);
13.
14.    asio::io_context  ios;
15.
16.    try {
17.       // Step 3. Instantiating and opening an acceptor socket.
18.       asio::ip::tcp::acceptor acceptor(ios, ep.protocol());
19.
20.       // Step 4. Binding the acceptor socket to the
21.       // server endpint.
22.       acceptor.bind(ep);
23.
24.       // Step 5. Starting to listen for incoming connection
25.       // requests.
26.       acceptor.listen(BACKLOG_SIZE);
27.
28.       // Step 6. Creating an active socket.
29.       asio::ip::tcp::socket sock(ios);
30.
31.       // Step 7. Processing the next connection request and
32.       // connecting the active socket to the client.
33.       acceptor.accept(sock);
34.
35.       // At this point 'sock' socket is connected to
36.       //the client application and can be used to send data to
37.       // or receive data from it.
38.    }
39.    catch (system::system_error& e) {
40.       std::cout << "Error occured! Error code = " << e.code()
41.          << ". Message: " << e.what();
42.
```

```
43.         return e.code().value();
44.     }
45. }
```

关于buffer

任何网络库都有提供buffer的数据结构，所谓buffer就是接收和发送数据时缓存数据的结构。
boost::asio提供了asio::mutable_buffer 和 asio::const_buffer这两个结构，他们是一段连续的空间，首字节存储了后续数据的长度。
asio::mutable_buffer用于写服务，asio::const_buffer用于读服务。但是这两个结构都没有被asio的api直接使用。
对于api的buffer参数，asio提出了MutableBufferSequence和ConstBufferSequence概念，他们是由多个asio::mutable_buffer和asio::const_buffer组成的。也就是说boost::asio为了节省空间，将一部分连续的空间组合起来，作为参数交给api使用。
我们可以理解为MutableBufferSequence的数据结构为std::vector<asio::mutable_buffer>
结构如下



每隔vector存储的都是mutable_buffer的地址，每个mutable_buffer的第一个字节表示数据的长度，后面跟着数据内容。

这么复杂的结构交给用户使用并不合适，所以asio提出了buffer()函数，该函数接收多种形式的字节流，该函数返回asio::mutable_buffers_1 o或者asio::const_buffers_1结构的对象。
如果传递给buffer()的参数是一个只读类型，则函数返回asio::const_buffers_1 类型对象。
如果传递给buffer()的参数是一个可写类型，则返回asio::mutable_buffers_1 类型对象。
asio::const_buffers_1和asio::mutable_buffers_1是asio::mutable_buffer和asio::const_buffer的适配器，提供了符合MutableBufferSequence和ConstBufferSequence概念的接口，所以他们可以作为boost::asio的api函数的参数使用。
简单概括一下，我们可以用buffer()函数生成我们要用的缓存存储数据。
比如boost的发送接口send要求的参数为ConstBufferSequence类型

```
1. template<typename ConstBufferSequence>
2. std::size_t send(const ConstBufferSequence & buffers);
```

我们需要将”Hello Word转化为该类型”

```
1. void use_const_buffer() {
2.     std::string buf = "hello world!";
3.     asio::const_buffer  asio_buf(buf.c_str(), buf.length());
4.     std::vector<asio::const_buffer> buffers_sequence;
5.     buffers_sequence.push_back(asio_buf);
6. }
```

最终buffers_sequence就是可以传递给发送接口send的类型。但是这太复杂了，可以直接用buffer函数转化为send需要的参数类型

```
1. void use_buffer_str() {
2.     asio::const_buffers_1 output_buf = asio::buffer("hello world");
3. }
```

output_buf可以直接传递给该send接口。我们也可以将数组转化为send接受的类型

```
1. void use_buffer_array(){
2.     const size_t  BUF_SIZE_BYTES = 20;
3.     std::unique_ptr<char[] > buf(new char[BUF_SIZE_BYTES]);
4.     auto input_buf = asio::buffer(static_cast<void*>(buf.get()), BUF_SIZE_BYTES);
5. }
```

对于流式操作，我们可以用streambuf，将输入输出流和streambuf绑定，可以实现流式输入和输出。

```
1. void use_stream_buffer() {
2.     asio::streambuf buf;
3.
4.     std::ostream output(&buf);
5.
6.     // Writing the message to the stream-based buffer.
7.     output << "Message1\nMessage2";
8.
9.     // Now we want to read all data from a streambuf
10.    // until '\n' delimiter.
11.    // Instantiate an input stream which uses our
12.    // stream buffer.
13.    std::istream input(&buf);
14.
15.    // We'll read data into this string.
16.    std::string message1;
17.
18.    std::getline(input, message1);
19.
20.    // Now message1 string contains 'Message1'.
21. }
```

浏览(7636) (JavaScript:void(0);) 评论(1) (JavaScript:void(0);) ♡喜欢(527) (JavaScript:void(0);)

热门评论

请输入正文

评论



lzz

2023-08-30 16:44:38

链接好像挂掉了。

♡ 喜欢(3) (JavaScript:void(0);) 回复(1) (JavaScript:void(0);)

共有1条回复, 点击查看 ⩔

没有更多评论了,点击收起 ⩓ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 👁浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 👁浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 👁浏览(1595)

解密定时器的实现细节

♡喜欢(566) 👁浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 👁浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满足

sort所需interface的规定即可，需实现Len, Swap, Less三个方法，只要实现了这三个方法都可以用来做sort排序的参数。

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)